I102 - Paradigmas de Programación

INTRODUCCIÓN

1

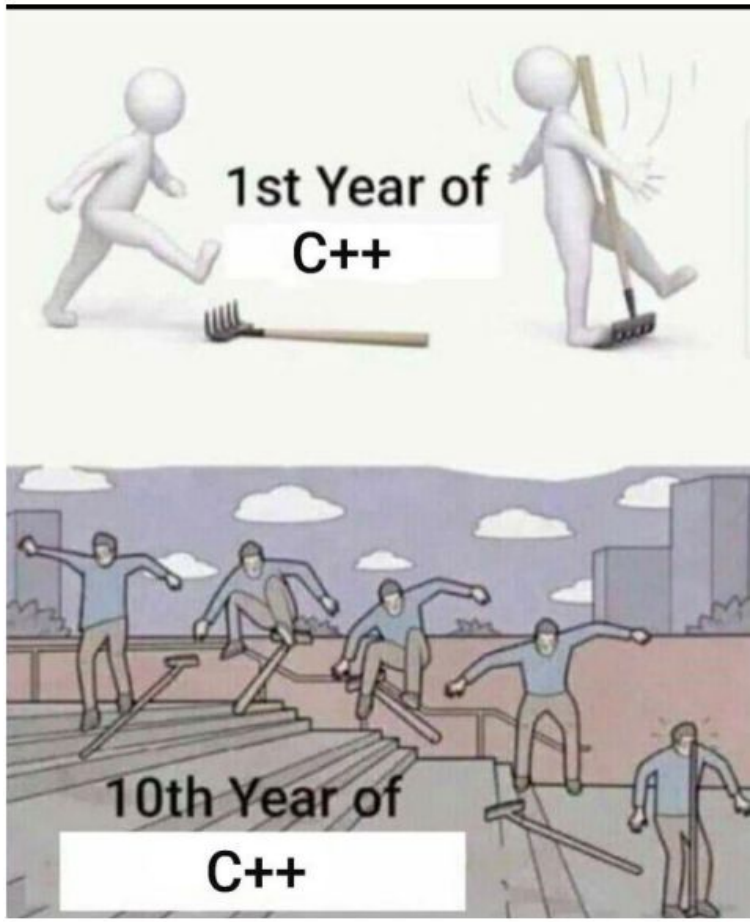
Prof. Mariano Scaramal & Javier Godoy
Ingeniería en Inteligencia Artificial

Contenidos

- Introducción al curso.
- Introducción a C++.
- El ambiente de trabajo de la materia.
  - Compilador de C++
  - Visual Studio Code
  - Revisión de Git
  - Comandos básicos de Linux

Aclaraciones sobre el curso…

- Enfoque en programar y plasmar el conocimiento adquirido (no todo será dado en clases).
- Manejo de C++ básico, OOP, librería standard, programación genérica y programación concurrente.
- El curso se enfocará mayormente en programación y diseño de código.
- No obstante, se incluirán contenidos de teoría de computación en los homeworks y trabajos prácticos.

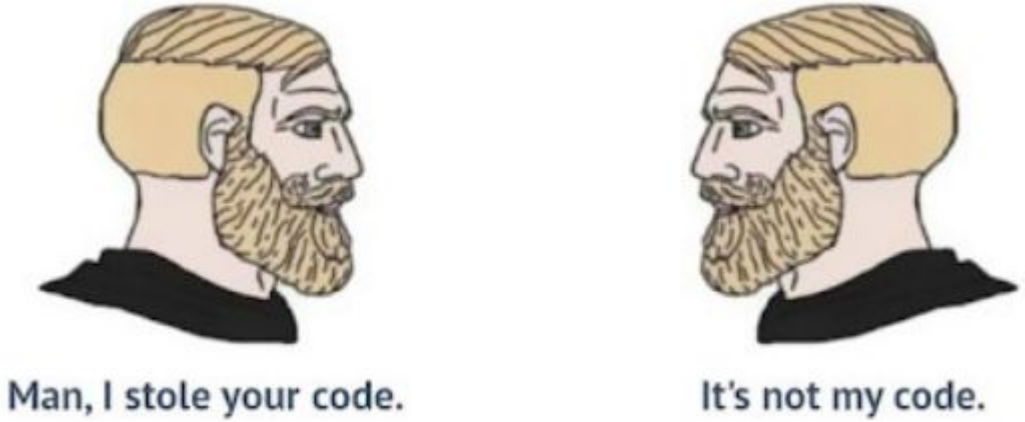


Al final del curso podrán…

- Diseñar e implementar software en C++.
- Desarrollar software fácil de leer, entender, y mantener.
- Entender partes de código provisto, analizar su salida e implementar posibles mejoras.
- Implementar código para resolver problemas específicos.
- Analizar código con el fin de encontrar y resolver bugs, problemas de eficiencia, o causas de fallos.
- Utilizar la librería standard de C++ e implementar las propias.

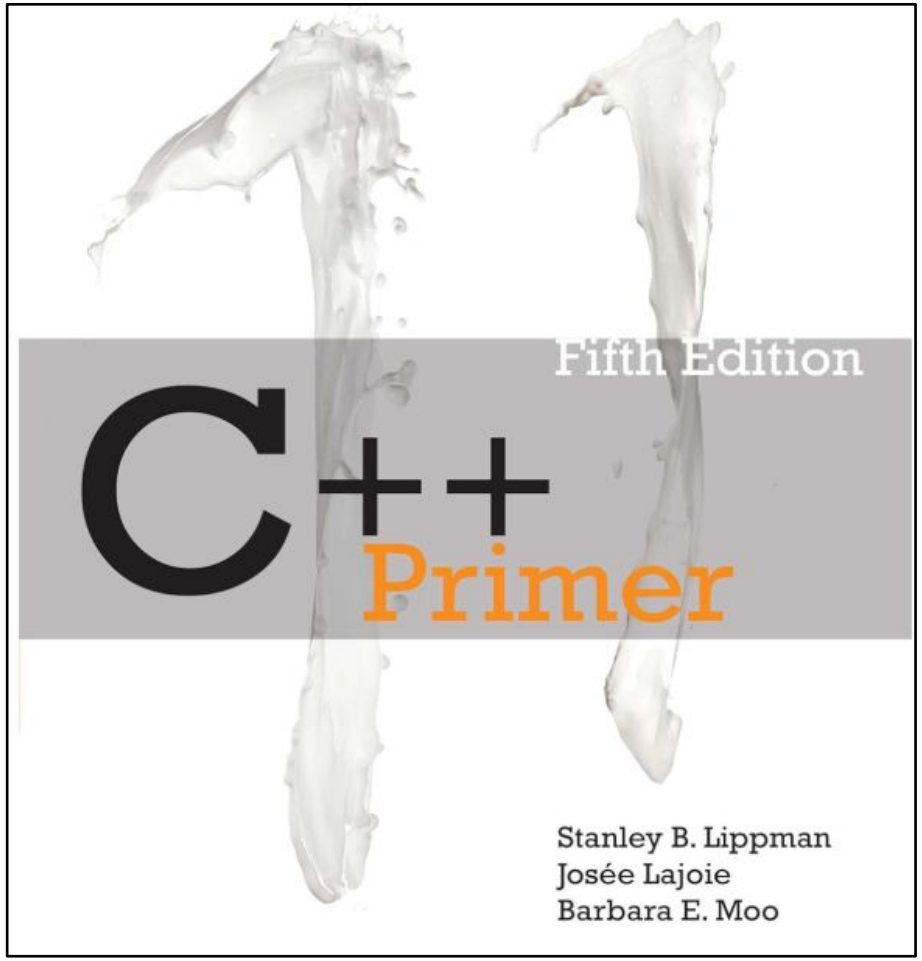
Copia, Plagio y Deshonestidad Intelectual

- Leer el apartado en el programa de la materia.
- Ante cualquier duda, siempre consultar y no asumir. De esta forma se evitan problemas.
- La colaboración entre alumnos es recomendada, no obstante, cada alumno debe presentar su propio código sin que sea similar a otro.
  - <u>Ejemplo importante</u>: Cambiar el nombre de las variables o el orden en que se definen las mismas no cuenta como presentar su propio código.



Bibliografía y Referencia

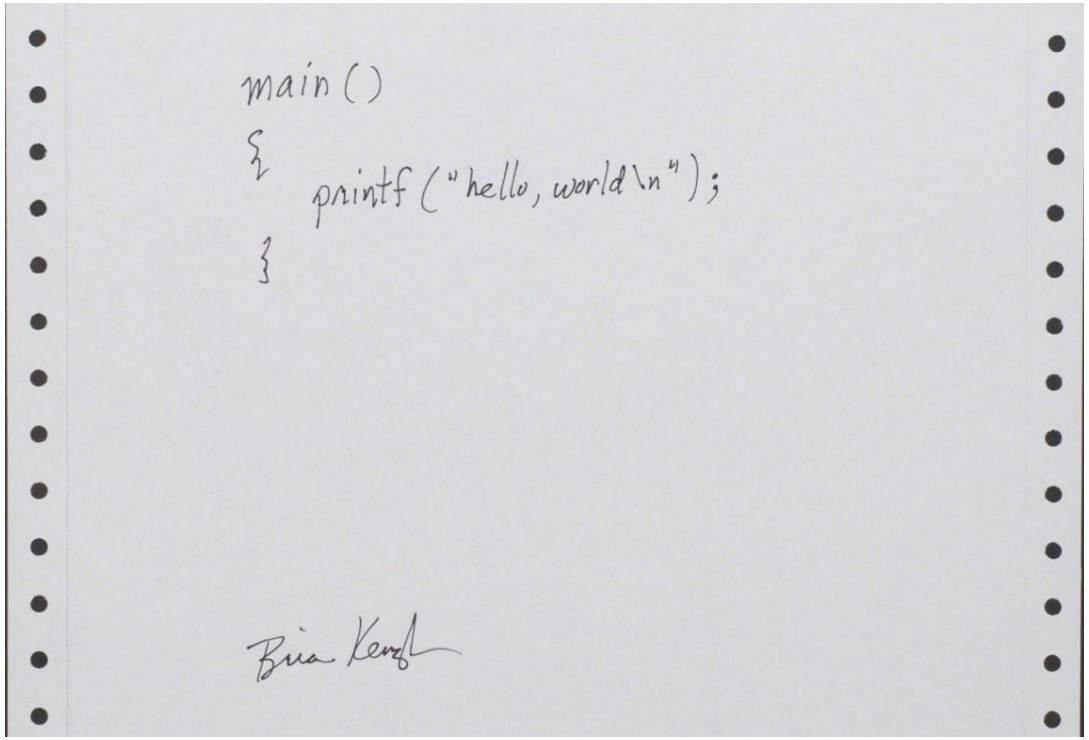
- La biblioteca posee una amplia gama de libros (en inglés y español) introductorios para programar en C++.
- El libro que mayormente seguirá la cátedra es “C++ Primer 5th Edition” de Stanley Lippman (tener en cuenta que hay libros con nombres similares).
- Otras bibliografías para programación concurrente y UML se pueden encontrar en el programa de la materia.
- La referencia oficial de C++ a consultar se encuentra en: cppreference.com



Bibliografía recomendada para la primera parte (amazon.com/Primer-5th-Stanley-B-Lippman/dp/0321714113)

Historia de C/C++ - El lenguaje C

- El lenguaje de programación C fue creado por Dennis Ritchie en Bell Labs al inicio de la década del 70.
- El lenguaje C evolucionó del lenguaje B, desarrollado en 1969 por Ken Thompson y Dennis Ritchie.
- C fue desarrollado para simplificar la programación en Unix.
- Se volvió muy popular al final de la década del 70 por la mayor mercado de computadoras.
- En 1989, C se vuelve un standard o norma conocida como ANSI C.



Código de “Hello, world” en C por Brian Kernighan (en.wikipedia.org/wiki/The_C_Programming_Language)

Historia de C++ - El lenguaje C++

- El lenguaje de programación C++ fue desarrollado por Bjarne Stroustrup al comienzo de la década del 80.
- C++ fue la tesis de doctorado de Bjarne Stroustrup (Bjarne Stroustrup's Homepage).
- Si bien no fue el primer lenguaje orientado a objetos, proveyó esta mejora a un lenguaje muy difundido como C.
- C++ influyó en el desarrollo de otros lenguajes como Java y C#.
- C++ es normalizado por ISO. Hasta la fecha se publicaron 7 revisiones y actualmente se publican cada 3 años.

C++ standards

Year	ISO/IEC Standard	Informal name
1998	14882:1998[41]	C++98
2003	14882:2003[42]	C++03
2011	14882:2011[43]	C++11, C++0x
2014	14882:2014[44]	C++14, C++1y
2017	14882:2017[45]	C++17, C++1z
2020	14882:2020[46]	C++20, C++2a
2024	14882:2024[17]	C++23, C++2b
TBA		C++26, C++2c

Normas de C++
(en.wikipedia.org/wiki/C++)

El lenguaje C++

- Tipado estático o dinámico?
- Tipado fuerte o débil?

El lenguaje C++

```
int main()
{
int valor = 10;
// ERROR DE COMPILACIÓN:
// error: invalid conversion from 'const char*' to 'int'
   valor = "Hola";

   return 0;
}
```

```
valor = 10      # 'valor' apunta a un objeto de tipo int
valor = "Hola"   # Totalmente válido. Ahora apunta a
un objeto de tipo str

print(valor)     # Salida: Hola
```

- C++: Tipado estático
- Python: Tipado dinámico

El lenguaje C++

```cpp
#include <iostream>
int main() {
    int numero = 5;
    char letra = 'A'; // En ASCII, 'A' es el valor 65

    // Válido. C++ promueve el 'char' a 'int'
silenciosamente.
    int resultado = numero + letra;
    std::cout << resultado << "\n"; // Salida: 70
    return 0;
}
```

- C++: Tipado Débil

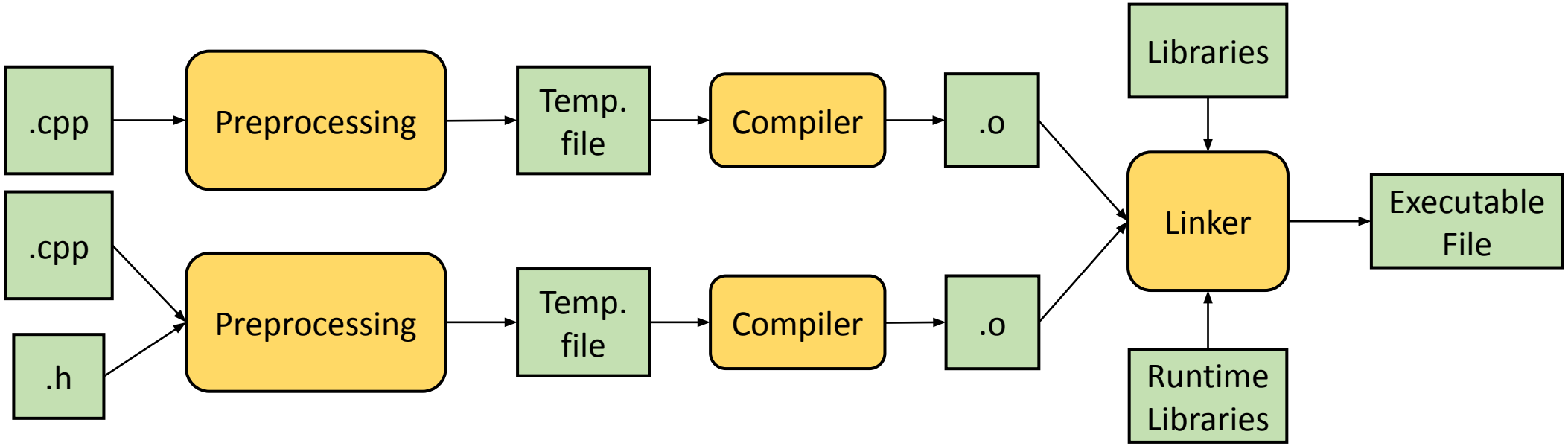
```python
numero = 5
letra = 'A'

# ERROR EN TIEMPO DE EJECUCIÓN:
# TypeError: unsupported operand type(s) for +: 'int'
and 'str'
resultado = numero + letra
```

- Python: Tipado Fuerte

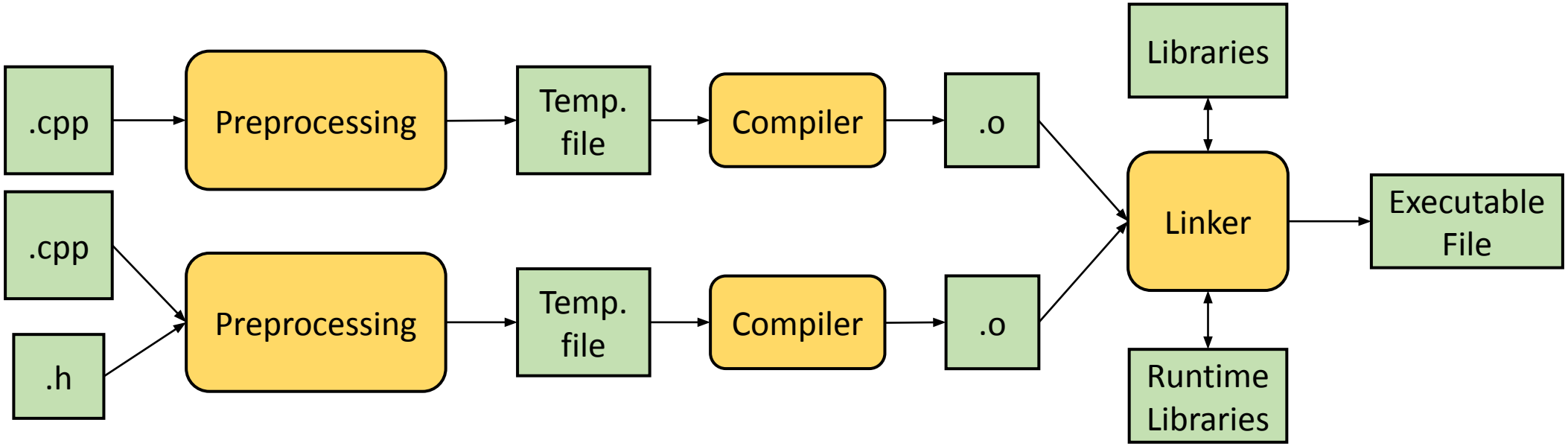
C++ Compiler y Linker

- El código en C++ debe ser transformado en un archivo ejecutable.
- Los archivos de C++ (.cpp, .h, etc) entran al preprocesador que genera un archivo temporal según los comandos de preprocesador (#define, #include, etc).
- Los archivos temporales van al compilador, produciendo un archivo binario .obj/.o (Win, Linux o Mac) con los comandos en assembler y un diccionario de las funciones y variables definidas.
- El LINKER combina archivos .obj/.o, librerías y el código necesario para crear el ejecutable.
- Errores pueden encontrarse tanto en el compilador (C<code>) como en el linker (LNK<code>).



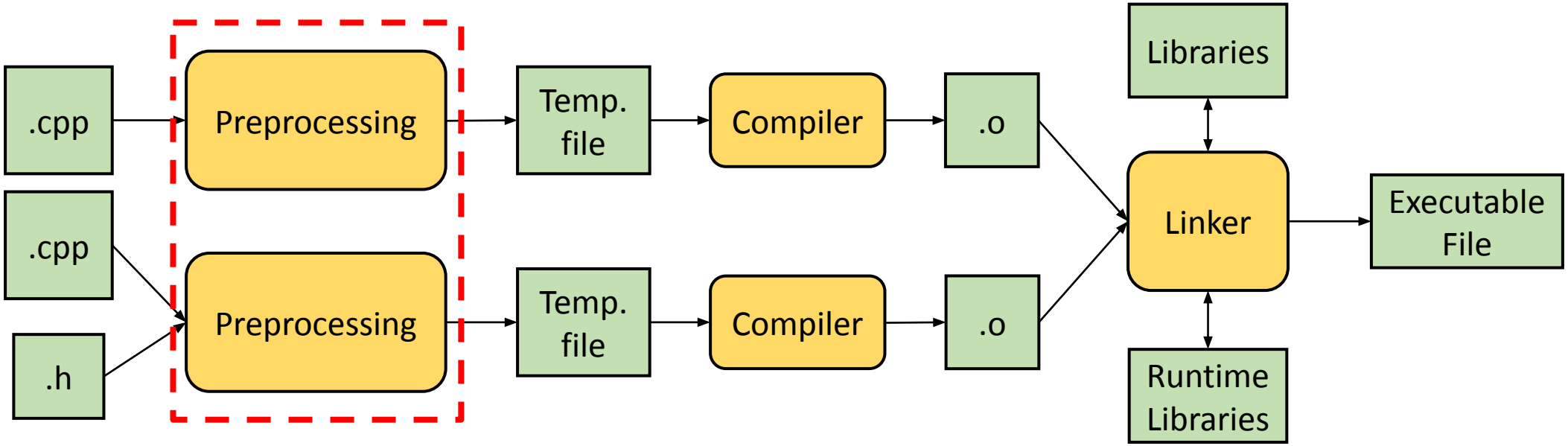
C++ Compiler y Linker

- El producto final no tiene que ser exclusivamente un archivo ejecutable.
- Existen librerías dinámicas y estáticas, utilizadas para distribuir funcionalidad entre proyectos.
- Se verá en su momento que las librerías estáticas (obtenidas por static linking) y dinámicas (dynamic linking) tienen sus ventajas y desventajas.
- Las librerías también son generadas por el sistema de compilación y linkeo.



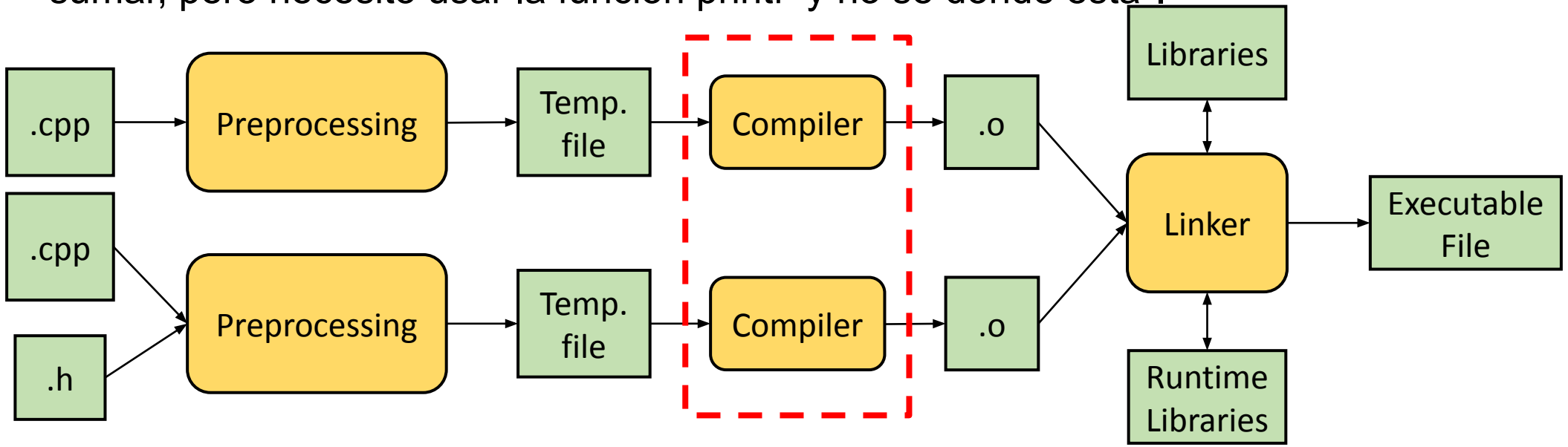
Preprocessing

- No sabe qué es C++ ni sabe sobre tipos de datos
- Básicamente se encarga de 2 tareas:
  - Expansión de Macros (#define): Reemplaza texto por texto antes de que el compilador vea el código.
  - Resolución de Inclusiones (#include): Copia y pega el contenido completo del archivo .h dentro del .cpp.



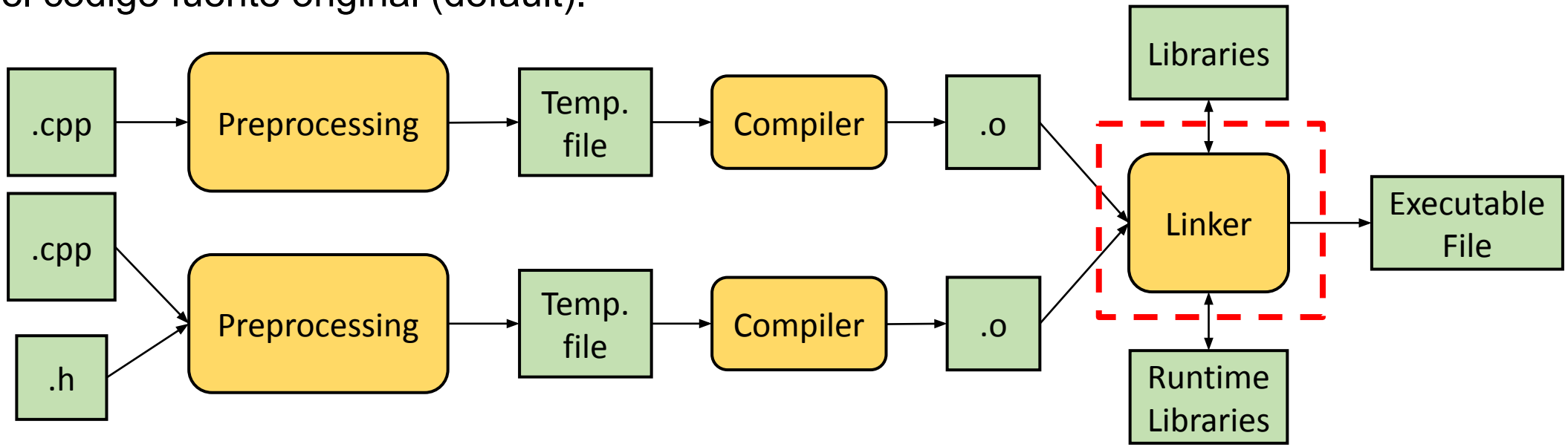
Compiler

- El input es el output del *Preprocessing.*
- Algunas de las operaciones que realiza son:
  - Análisis sintáctico: Convierte el texto en un AST.
  - Análisis semántico: Se verifican las reglas de tipos, entre otros.
  - Código Objeto (.o/.obj): El resultado no es un ejecutable aún. Es código de máquina (instrucciones de CPU) con "huecos".
  - Tabla de Símbolos: cada archivo .o incluye un "diccionario": "Aquí implemento la función sumar, pero necesito usar la función printf y no sé dónde está".



Linker

- El input es el output del *Compiler.*
- Algunas de las operaciones que realiza son:
  - Resolución de Símbolos: Tapa los “huecos”: si el archivo A.o llama a printf, busca la implementación binaria de printf en la librería estándar de C y los conecta.
  - Binario Final: Empaqueta todo siguiendo un formato específico del sistema operativo, generando un ejecutable independiente (.exe, .out) o una librería dinámica (.dll, .so)
- Recordar: el ejecutable final pierde los nombres de las variables y las referencias a las líneas del código fuente original (default).



El compilador g++

- g++ es un compilador diseñado para C++, parte del GNU Compiler Collection (GCC).
- g++ puede compilar y linkear un proyecto automáticamente.
- Algunas opciones/flags de g++ son:
  - -c: solo compilar, devuelve los archivos de objetos (.obj/.o).
  - -o: especifica el archivo de salida (ejecutable o objeto, default “a.out”)
  - -Wall: muestra casi todos los mensajes de warnings.
  - -Wextra: muestra todos los mensajes de warnings.
  - -std=<standard>: especifica la norma de C++ (-std=c++11, -std=c++17, etc)
  - -g: habilita información de debugging (para usar con debuggers como gdb)
  - -I <folder>: (i mayus) agrega directorios para incluir header files búsquedas (preprocessor)
  - -L<folder>: agregar directorios para incluir librerías (linker)
  - -l<file>: se indica la librería (estática o dinámica) a linkear en el folder indicado por -L
  - -lstdc++: linkea el proyecto (usualmente no necesario dado que se hace automáticamente)

- Ejemplo donde se define el include folder que contiene los header files y/o librerías:

  g++ -g *.cpp –I ./include –o my_Project

- Proyectos grandes suele utilizar sistemas de compilación como Make o Cmake

GNU Debugger o gdb

- Otra herramienta de GNU es el debugger **gdb**
- Sólo se puede utilizar si se compila el código con la opción –g (información de debugging)
- Ejemplo:
  - gdb ./HelloWorld.cpp
- Lista de algunos comandos:
  - **b <nro linea>:** agrega un breakpoint
  - **info b:** lista de breakpoints
  - **del <nro breakpoint>:** borra un breakpoint
  - **info locals:** variables en el scope actual.
  - **p <nombre var>:** imprime el valor de la variable
    - **p/x <nombre var>:** imprime el contenido de la variable x en hexadecimal, otras opciones son /d, /f, /c, /s, etc)
  - **next (n) y step (s):** son ir a la siguiente instrucción y “step into”, respectivamente.
  - **run (r) y continue (c):** ejecuta el código a analizar y continua la ejecución luego de llegar a un breakpoint, respectivamente.
  - **display <nombre var>**: Imprime el valor de var automáticamente cada vez que el programa se pausa
  - **quit (q):** sale de gdb.

GNU Debugger o gdb

- Modo TUI (Text User Interface)
  - GDB por defecto es solo texto
  - Al utilizar a un modo gráfico
  - Ejemplo gdb

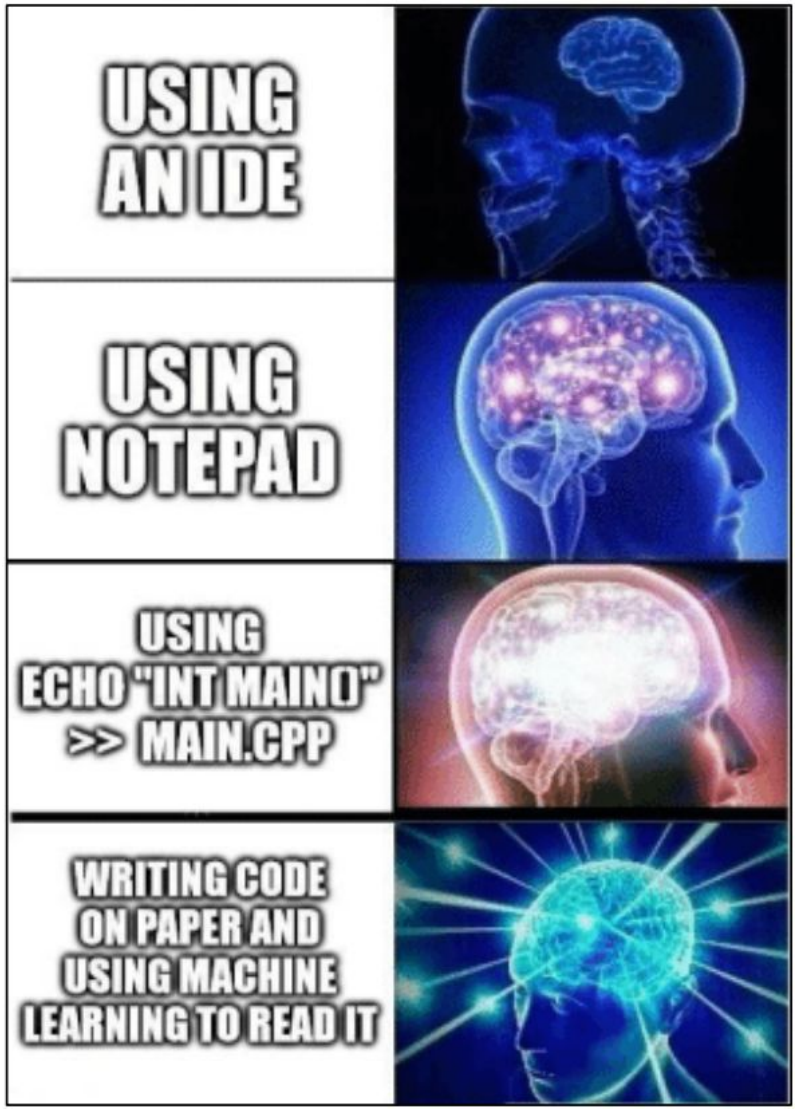
- Recomendado: usar de VS Code. 💥

```
hello.cpp
        1  #include <iostream>
        2  #include <typeinfo>
        3
        4  int main()
        5  {
b+      6      int a = 5;
        7      char b = '2'; // Internamente representado por el valor ASCII 50
        8
        9      // El compilador permite la operación y deduce el tipo:
       10      auto resultado = a + b;
       11
       12      std::cout << "Resultado: " << resultado << "\n";
       13      std::cout << "Tipo deducido: " << typeid(resultado).name() << "\n";
       14
       15      // Salida:
       16      // Resultado: 55
       17      // Tipo deducido: i (entero)
exec No process In:
--Type <RET> for more, q to quit, c to continue without paging--show <https://www.gnu.org/soft
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from hello_world...
(gdb) b 6
```

Integrated Developement Environment (IDE)

- La opción del IDE suele ser propia del equipo de desarrollo o de cada desarrollador.
- Una opción popular es Visual Studio Code que puede ser utilizado en Linux, Win y MacOs.
- Primero, instalar g++ y gdb:
  - sudo apt-get update
  - sudo apt-get install g++
  - sudo apt-get install gdb
- Instalar VS Code para Linux
- Una vez instalado VS Code, instalar el paquete “C/C++ Extension Pack”.



Distributed Version Control System - GitHub

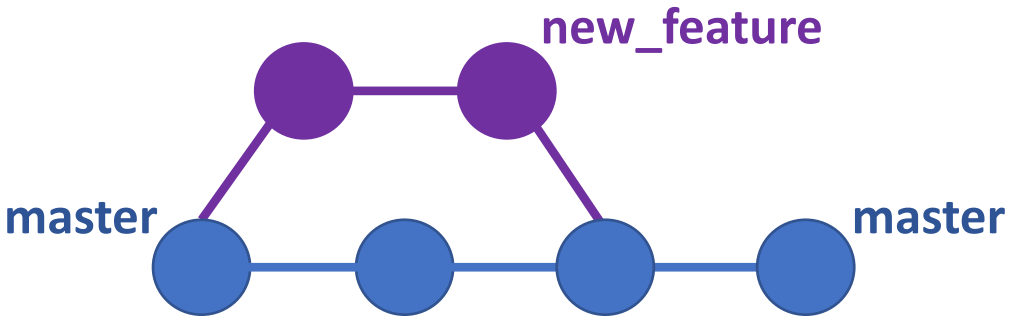
- Un dVCS es un software que permite el control y seguimiento de archivos relacionados a un proyecto (ya visto en “Pensamiento Computacional”).
- Es indispensable para el desarrollo de un proyecto y también para crear distintas versiones de los mismos según requerimientos o clientes.
- Revisar documento “Git Basics” de “Pensamiento Computacional” (en moodle).
- Descargar hoja de comandos Git en Git cheat sheet | Atlassian Git Tutorial. Este link también ofrece un instructivo con videos gratuitos.
- Para mantener el orden de los proyectos, los commits deben tener un mensaje adecuado. Usualmente se utiliza el número y el título del ticket que se resuelve. En la materia, se usa una descripción representativa de los cambios.
- Tener en cuenta que los homeworks de la materia deberán ser entregados mediante git.

Algunos comandos útiles

- Crear un branch local y seleccionarlo: ***git checkout –b <branchName>***
- Cambiar de branch: ***git checkout <branchName>***
- Ver en que branch se encuentra: ***git branch***
- Ver el estado de los cambios: ***git status***
- Agregar archivos para el commit: ***git add <directory/files>***
- Agrega archivos de forma local con msj **git commit -m "mensaje" <files>**
- Obtiene una copia del branch especifico: ***git pull <remote>***
- Actualiza el branch con el nuevo contenido: ***git push <remote> <branch>***
- Descarga un remote branch específico: ***git fetch <remote> <branch>***
- Diferencias entre local y remote: ***git diff HEAD***
- Muestra el log de los ultimo n commits: ***git log –n***

El comando ***merge***

En el repositorio local:



- Ir al branch **master** y ejecutar:

git merge new_feature

- Resolver los conflictos y hacer el *push* de **master** branch al repositorio remoto.
- Esta no es la forma en que se realiza usualmente un *merge.*
- Usualmente, se crea un *pull request* desde GitHub y si se aprueba, se hace el merge del branch **new_feature** a master automáticamente. Esto se hace desde GitHub luego de que se hace un push al repositorio remoto.

Terminal de Linux

- La terminal de Linux puede proporcionar una gran cantidad de información. Como se observa en el siguiente link Linux Commands.

Comando	Descripción	Ejemplo
cd	Cambiar directorio a un destino	cd /path/directoriyNme
clear	Borrar pantalla (análogo a ´cls´ en Windows)	clear
cp	Copiar archivo/s	cp /path/fileName1 /path2/fileName1
diff	Compara el contenido de archivos	diff fileName1 fileName2
exit	Salir de linux (análogo a ´exit´ en Windows)	exit
grep	Encuentra un string en un archivo de texto	grep “my string” fileName
head	Muestra las primeras líneas de un archivo	head fileName
kill / killall	Terminar un proceso, Siñales: -15 (default), el proceso tiene la oportunidad de terminar tareas de limpieza, -2, el proceso puede no terminar tareas de limpieza. -9, el proceso es terminado en forma inmediata	kill -9 <PI> killall -9 firefox

Terminal de Linux (cont'd)

Comando	Descripción	Ejemplo
less	Muestra el contenido de un archivo (´q´ par salir)	less fileName
ls	Muestra el contenido de un directorio	ls /path/directoryName
mv	Mueve o renombra archivo/s	mv /path/fileName1 /path2/fileName2
mkdir	Crea un directorio	mk /path/directoryName
ps	Muestra los procesos en ejecución	ps -ef
pwd	Muestra el directorio actual de trabajo	pwd
rm	Borra archivo/s	rm fileName
rmdir	Borra un directorio	rm directoryName
sudo	Ejecutar comandos con privilegios de root user	sudo <comando> (requiere password)
tail	Muestra las últimas líneas de un archivo	tail fileName
tar	Combina archivos con opción de compresión	Tar fileName
touch	Crea un archivo de texto (si el mismo no existe)	touch /path/fileName
vi	Editor de texto muy simple	vi fileName
whatis	Muestra información sobre un comando de Linux	whatis sudo

Ejemplos de Terminal de Linux

Como terminar un proceso en LINUX:

1. Buscar el process id (PI) del proceso a terminar con el comando:

   ps -ef

2. Terminar el proceso con:

   kill -15,2,9 PI

Los comandos tienen distintos flags como es el caso de:

ls –la

–l: muestra información de los archivos,
–a: que muestre todos los archivos, incluyendo archivos ocultos.

Preguntas?